

GCP AI/ML Interview Questions (50 Questions)

Core GCP ML Services (Questions 1-10)

Q1: What is Vertex AI and how does it unify GCP's ML services?

Answer: Vertex AI is GCP's unified ML platform that consolidates AI Platform, AutoML, and other ML services.

Key Components:

- **Vertex AI Workbench:** Managed Jupyter notebooks
- **Vertex AI Training:** Custom model training
- **Vertex AI Prediction:** Model deployment
- **Vertex AI Pipelines:** MLOps workflows (Kubeflow)
- **Vertex AI Feature Store:** Centralized feature management
- **Vertex AI Model Registry:** Model versioning
- **Vertex AI Experiments:** Tracking and comparison
- **Vertex AI Metadata:** Lineage tracking
- **AutoML:** No-code model training
- **Vertex AI Matching Engine:** Vector similarity search

Unified Benefits:

- Single API for all ML operations
- Consistent interface across services
- Integrated MLOps
- Pre-built models and custom training

Example:

```
from google.cloud import aiplatform

aiplatform.init(project='my-project', location='us-central1')

# Train custom model
job = aiplatform.CustomTrainingJob(
    display_name='training-job',
    script_path='train.py',
    container_uri='gcr.io/cloud-aiplatform/training/pytorch-gpu.1-9:latest',
    model_serving_container_image_uri='gcr.io/cloud-aiplatform/prediction/pytorch-gpu.1-9:latest'
)

model = job.run(
    dataset=dataset,
    replica_count=1,
    machine_type='n1-standard-4',
    accelerator_type='NVIDIA_TESLA_T4',
    accelerator_count=1
```

```
)

# Deploy
endpoint = model.deploy(machine_type='n1-standard-4')
```

Q2: Compare Vertex AI AutoML vs Custom Training.

Answer:

Vertex AI AutoML:

- No-code ML model building
- Automatic feature engineering
- Neural Architecture Search (NAS)
- Hyperparameter tuning included
- Best for: Tabular, image, text, video data
- Easy to use, minimal ML expertise
- Limited customization

Custom Training:

- Full control over model architecture
- Any framework (TensorFlow, PyTorch, Scikit-learn)
- Custom preprocessing
- Best for: Complex models, research, fine-tuning
- Requires ML expertise
- More flexible

Aspect	AutoML	Custom Training
Code Required	Minimal	Extensive
Expertise	Low	High
Time	Hours	Days/Weeks
Customization	Limited	Full
Cost	Higher per model	Lower
Best For	Standard tasks	Complex/specialized

Example AutoML:

```
# AutoML Tabular
dataset = aiplatform.TabularDataset.create(
```

```
    display_name='customer-churn',
    gcs_source='gs://bucket/data.csv'
)

job = aiplatform.AutoMLTabularTrainingJob(
    display_name='churn-automl',
    optimization_prediction_type='classification',
    optimization_objective='maximize-au-roc'
)

model = job.run(
    dataset=dataset,
    target_column='churned',
    training_fraction_split=0.8,
    validation_fraction_split=0.1,
    test_fraction_split=0.1,
    budget_milli_node_hours=1000 # Max 1 node-hour
)
```

Q3: What is Vertex AI Workbench and its types?

Answer:

Vertex AI Workbench provides managed Jupyter notebook environments.

Two Types:

1. Managed Notebooks (Recommended):

- Fully managed, serverless
- Automatic idle shutdown
- Pre-installed ML frameworks
- Integrated with Vertex AI services
- Better security (VPC-SC support)
- Pay only when running

2. User-Managed Notebooks (Legacy):

- More control over VM
- Custom configurations
- Requires manual management
- Always running (unless stopped)

Features:

- Pre-built containers (TensorFlow, PyTorch, scikit-learn)
- GPU support
- BigQuery integration
- Git integration
- Scheduling notebooks

Example:

```
# Create managed notebook
from google.cloud import notebooks_v1

client = notebooks_v1.ManagedNotebookServiceClient()

notebook = notebooks_v1.Runtime(
    display_name='ml-notebook',
    access_config=notebooks_v1.RuntimeAccessConfig(
        access_type=notebooks_v1.RuntimeAccessConfig.RuntimeAccessType.SINGLE_USER
    ),
    software_config=notebooks_v1.RuntimeSoftwareConfig(
        notebook_upgrade_schedule='',
    ),
    virtual_machine=notebooks_v1.VirtualMachine(
        virtual_machine_config=notebooks_v1.VirtualMachineConfig(
            machine_type='n1-standard-4',
            data_disk=notebooks_v1.LocalDisk(
                initialize_params=notebooks_v1.LocalDiskInitializeParams(
                    disk_size_gb=100,
                    disk_type=notebooks_v1.DiskType.PD_STANDARD
                )
            )
        )
    )
)
```

Q4: Explain Vertex AI Pipelines and Kubeflow integration.

Answer:

Vertex AI Pipelines orchestrates ML workflows using Kubeflow Pipelines (KFP).

Key Features:

- DAG-based workflow
- Component reusability
- Automatic logging and lineage
- Caching for faster iterations
- Scheduling and triggering
- Artifact tracking

Components:

- Python function-based
- Container-based
- Pre-built Google components

Example Pipeline:

```

from kfp.v2 import dsl, compiler
from kfp.v2.dsl import component, Input, Output, Dataset, Model

@component(
    base_image='python:3.9',
    packages_to_install=['pandas', 'scikit-learn']
)
def preprocess_data(
    input_data: Input[Dataset],
    output_data: Output[Dataset]
):
    import pandas as pd
    df = pd.read_csv(input_data.path)
    # Preprocessing logic
    df.to_csv(output_data.path, index=False)

@component
def train_model(
    training_data: Input[Dataset],
    model: Output[Model],
    learning_rate: float = 0.01
):
    # Training logic
    pass

@dsl.pipeline(
    name='ml-pipeline',
    description='End-to-end ML pipeline'
)
def pipeline(
    project_id: str,
    data_path: str
):
    preprocess_task = preprocess_data(input_data=data_path)

    train_task = train_model(
        training_data=preprocess_task.outputs['output_data'],
        learning_rate=0.01
    )

# Compile and run
compiler.Compiler().compile(
    pipeline_func=pipeline,
    package_path='pipeline.json'
)

# Execute
from google.cloud import aiplatform

job = aiplatform.PipelineJob(
    display_name='ml-pipeline-run',
    template_path='pipeline.json',

```

```
        parameter_values={
            'project_id': 'my-project',
            'data_path': 'gs://bucket/data.csv'
        }
    )

    job.run()
```

Q5: What is Vertex AI Feature Store and its benefits?

Answer:

Vertex AI Feature Store manages, serves, and shares ML features.

Architecture:

- **Featurestore:** Top-level container
- **EntityType:** Category of entities (user, product)
- **Feature:** Individual feature (age, price)

Two Serving Modes:

1. **Online:** Low-latency (<10ms) for real-time predictions
2. **Offline:** Batch retrieval for training

Benefits:

- Feature reusability across models
- Consistent features (train/serve)
- Point-in-time correctness (avoid data leakage)
- Feature versioning
- Monitoring feature distribution

Example:

```
# Create Feature Store
featurestore = aiplatform.Featurestore.create(
    featurestore_id='customer_features',
    online_serving_config=aiplatform.featurestore.OnlineServingConfig(
        fixed_node_count=1
    )
)

# Create entity type
entity_type = featurestore.create_entity_type(
    entity_type_id='customer',
    description='Customer features'
)

# Create features
```

```

entity_type.create_feature(
    feature_id='age',
    value_type='INT64'
)

entity_type.create_feature(
    feature_id='total_purchases',
    value_type='DOUBLE'
)

# Ingest features
entity_type.ingest_from_gcs(
    feature_ids=['age', 'total_purchases'],
    feature_time='timestamp',
    gcs_source_uris=['gs://bucket/features.csv']
)

# Online read
features = entity_type.read(
    entity_ids=['customer_123'],
    feature_ids=['age', 'total_purchases']
)

# Batch read for training
features_df = entity_type.batch_serve_to_df(
    read_instances_df=pd.DataFrame({'customer_id': ['c1', 'c2']}),
    start_time=datetime(2023, 1, 1),
    end_time=datetime(2023, 12, 31)
)

```

Q6-10: Quick Core Concepts

Q6: What is Vertex AI Matching Engine?

Vector similarity search service. Powered by Google's ScaNN algorithm. Use cases: Recommendation systems, semantic search, image similarity. Scales to billions of vectors.

Q7: Explain Vertex AI Model Monitoring.

Detects training-serving skew and prediction drift. Automatic alerting. Monitors: Feature distribution, prediction distribution. Integrated with Cloud Monitoring.

Q8: What are Vertex AI Experiments?

Track and compare ML experiments. Log parameters, metrics, artifacts. Compare runs side-by-side. Integration with TensorBoard.

Q9: Vertex AI Explainability methods?

- Sampled Shapley (feature attributions)
- Integrated Gradients (neural networks)

- XRAI (vision models) Explains individual predictions and global feature importance.

Q10: **What is Vertex AI Model Registry?**

Central repository for model versions. Tracks model lineage, evaluation metrics. Supports model approval workflow. Integrates with CI/CD.

GCP AI Services (Questions 11-20)

Q11: Compare Vision AI, Video Intelligence, and Cloud Document AI.

Answer:

Vision AI:

- Image analysis
- Object detection, face detection, OCR
- Label detection, logo detection
- Landmark recognition
- AutoML Vision for custom models

Video Intelligence:

- Video analysis
- Shot detection, object tracking
- Explicit content detection
- Speech transcription from video
- AutoML Video for custom models

Document AI:

- Extract structured data from documents
- Pre-built parsers: Invoice, Receipt, ID, Form
- Custom extractors
- Layout analysis, table extraction

Q12-50: GCP ML Services Quick Reference

Q12: Natural Language AI capabilities? Entity analysis, sentiment analysis, syntax analysis, content classification. AutoML Natural Language for custom models.

Q13: Speech-to-Text features? 125+ languages, streaming recognition, speaker diarization, automatic punctuation, word-level timestamps.

Q14: Text-to-Speech features? 220+ voices, 40+ languages, WaveNet and Neural2 voices, SSML support, custom voices.

Q15: Translation AI options? Basic vs Advanced. 100+ languages. AutoML Translation for custom models. Glossary support.

Q16: Dialogflow CX vs ES? ES: Simple bots. CX: Enterprise-scale, visual flow builder, state machines, A/B testing.

Q17: Recommendations AI use cases? E-commerce product recommendations. "Others you may like", "Frequently bought together", personalized homepage.

Q18: Contact Center AI components? Agent Assist, Insights, Virtual Agents (Dialogflow). Real-time agent suggestions.

Q19: BigQuery ML capabilities? Train models in SQL. Linear regression, logistic regression, K-means, AutoML, imported TensorFlow models.

Q20: Vertex AI Vizier? Black-box optimization service. Hyperparameter tuning. Bayesian optimization. Multi-objective optimization.

Q21: What is TPU and when to use? Tensor Processing Unit. Custom ASIC for ML. 10x faster than GPUs for large models. Use: Training large transformers, CNNs.

Q22: Vertex AI Tensorboard integration? Built-in Tensor Board. Track experiments, visualize metrics, compare runs. Automatic logging.

Q23: Vertex AI Metadata Store? Tracks ML metadata. Artifacts, executions, contexts. Lineage tracking. Query with ML Metadata API.

Q24: Pre-built containers in Vertex AI? TensorFlow, PyTorch, Scikit-learn, XGBoost. GPU and CPU variants. Prediction and training containers.

Q25: Vertex AI Model deployment options?

- Standard endpoints (dedicated resources)
- Private endpoints (VPC)
- Shared endpoints (lower cost)
- Batch prediction

Q26: Vertex AI Prediction pricing models? Per-node hour pricing. Auto-scaling support. Minimum 1 node. GPU pricing separate.

Q27: Deep Learning VM Images? Pre-configured VMs with ML frameworks. Jupyter, CUDA, cuDNN. One-click deployment. CPU and GPU variants.

Q28: Deep Learning Containers? Docker containers with ML frameworks. TensorFlow, PyTorch, scikit-learn. CPU and GPU optimized.

Q29: AI Platform Notebooks vs Workbench? Notebooks is legacy. Workbench is unified, managed, better integration with Vertex AI.

Q30: Vertex AI custom training with GPUs? Support for NVIDIA T4, P4, P100, V100, A100. Specify `accelerator_type` and `accelerator_count`.

Q31: Vertex AI Batch Prediction? Offline predictions on large datasets. Input/output from GCS or BigQuery. Cost-effective for bulk scoring.

Q32: Vertex AI Model Monitoring alerts? Cloud Monitoring integration. Email, SMS, PagerDuty alerts. Custom alert policies.

Q33: Vertex AI Private Endpoints? Deploy models without public IP. VPC access only. Enhanced security. Private Service Connect.

Q34: Vertex AI continuous evaluation? Monitor model performance over time. Compare against baseline. Detect degradation.

Q35: AI Hub (deprecated) replaced by? Vertex AI Model Garden. Pre-trained models, pipelines, notebooks. One-click deployment.

Q36: Vertex AI Fair AI practices? What-If Tool integration. Fairness Indicators. Explainable AI. Model Cards for transparency.

Q37: Vertex AI MLOps maturity levels? Level 0: Manual. Level 1: Automated training. Level 2: Automated CI/CD pipelines.

Q38: Vertex AI IAM roles?

- Vertex AI User: Full access
- Vertex AI Administrator: Admin operations
- Vertex AI Viewer: Read-only access
- Custom roles: Fine-grained permissions

Q39: Vertex AI quotas and limits? Request per minute, concurrent training jobs, endpoint QPS. Vary by region. Can request increase.

Q40: Vertex AI model versioning? Multiple model versions on same endpoint. Traffic splitting. A/B testing. Gradual rollout.

Q41: Vertex AI integration with Cloud Build? CI/CD pipelines for ML. Automated training on code commit. Model deployment automation.

Q42: Vertex AI integration with BigQuery? Direct data access. BigQuery as data source. Export predictions to BigQuery. BigQuery ML integration.

Q43: Vertex AI SDK languages? Python (primary), Java, Node.js. REST API for all languages.

Q44: Vertex AI training with custom containers? Bring your own container. Use any framework. Specify entry point script.

Q45: Vertex AI hyperparameter tuning? Built-in HP tuning. Bayesian optimization, grid search, random search. Parallel trials.

Q46: Vertex AI early stopping? Stop training jobs that won't improve. Save costs. Automatic or custom criteria.

Q47: Vertex AI checkpointing? Save training state. Resume from checkpoint. Use with preemptible VMs.

- Q48: Vertex AI preemptible VMs?** Up to 80% cost reduction. Training only (not prediction). Automatic restart with checkpointing.
- Q49: Vertex AI model export formats?** TensorFlow SavedModel, PyTorch, scikit-learn pickle, XGBoost, Custom artifacts.
- Q50: Vertex AI migration from AI Platform?** Use migration guides. API changes minimal. Managed notebooks auto-migrate. Training jobs require code updates.

Summary: GCP ML Services Comparison

Service	Purpose	Use Case
Vertex AI	Unified ML platform	Full ML lifecycle
AutoML	No-code ML	Quick model building
Vision AI	Image analysis	Object detection, OCR
Video Intelligence	Video analysis	Content moderation
Natural Language	Text analysis	Sentiment, entities
Translation	Language translation	Multi-language content
Speech-to-Text	Transcription	Voice recognition
Text-to-Speech	Voice synthesis	Accessibility
Dialogflow	Chatbots	Customer service
Recommendations AI	Product recommendations	E-commerce
Document AI	Document extraction	Invoice processing
BigQuery ML	SQL-based ML	Data analysts

Key Advantages:

- Unified platform (Vertex AI)
- Strong integration with BigQuery
- TPU availability
- AutoML for quick prototyping
- Scalable infrastructure